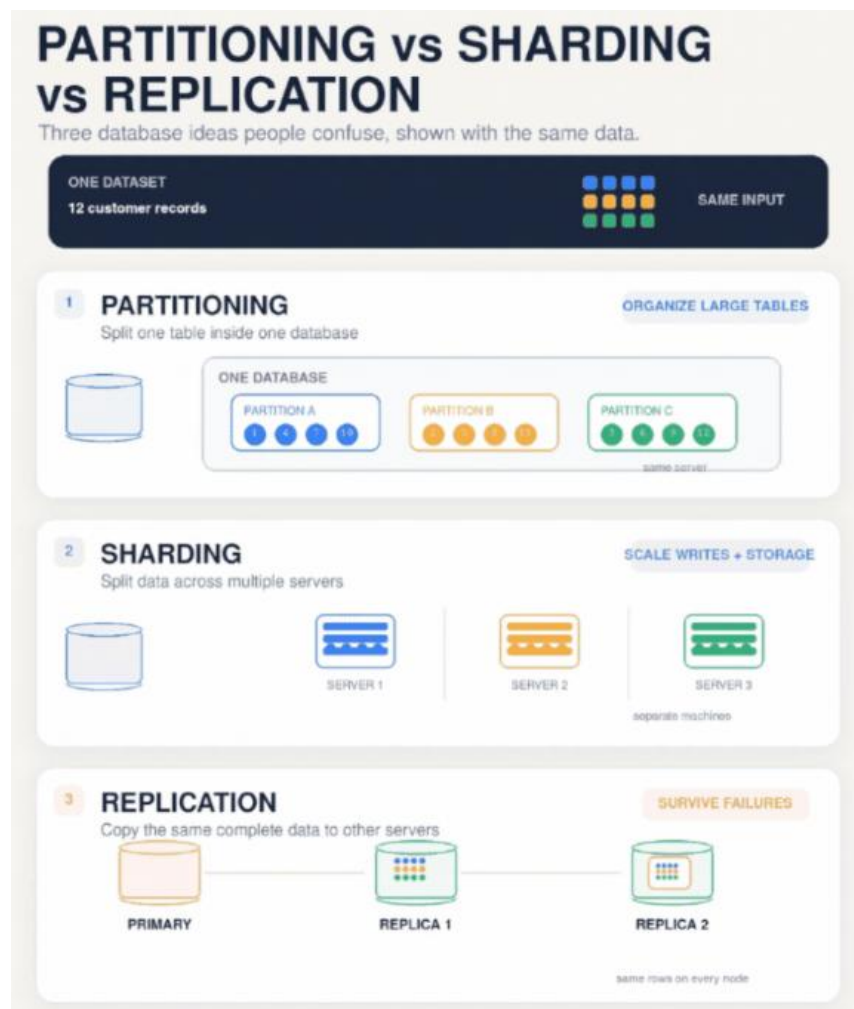


POSTGRESQL

Partitioning vs Replication vs Sharding

Technical Architecture & DBA Decision Guide

Scenario: 4 TB e-commerce order platform with increasing writes, frequent recent-order analytics, and a requirement to remain available after a database-node failure.



Original Concept & Image Credit

Shubham Srivastava

Principal Data Engineer @ Microsoft CoreAI | ex-Amazon | Data Engineering

[LinkedIn Profile](#) | [Original Post](#)

The original Partitioning vs Sharding vs Replication concept and supplied image are credited to Shubham Srivastava.

This document expands the topic with PostgreSQL-specific technical validation and DBA architecture guidance.

Validated against PostgreSQL 18 official documentation

PostgreSQL Partitioning | Replication | Sharding • Technical Reference

Index

Section	Topic
1	Executive Summary
2	The Interview Scenario
3	Partitioning - Managing Very Large Tables
4	Replication - High Availability and Read Scale
5	Physical vs Logical Replication
6	Sharding - Horizontal Distribution Across Servers
7	Choosing a Shard Key
8	Operational Cost of Sharding
9	Partitioning vs Sharding
10	Sharding vs Replication
11	Recommended Combined Architecture
12	Production Evolution Path
13	PostgreSQL DBA Decision Matrix
14	Five Architecture Questions
15	Interview-Ready Answer
16	PostgreSQL-Specific Accuracy Notes
17	Credits and Attribution
18	Official PostgreSQL References

Document purpose: Provide a production-oriented explanation of when PostgreSQL partitioning, replication, and sharding should be used, what each solves, and where their boundaries are.

1. Executive Summary

Partitioning, replication, and sharding are not competing answers to the same database problem. They solve different bottlenecks and can be combined in a mature PostgreSQL architecture.

Technology	Primary Goal	Typical Scope	Key Message
Partitioning	Large-table manageability & pruning	Within a PostgreSQL database	Organizes data
Replication	Availability & read scaling	Primary + standby/subscriber	Copies data
Sharding	Horizontal storage/write distribution	Multiple database nodes	Distributes data/workload

Core Principle: Start with the bottleneck. Do not introduce sharding simply because a database is large.

2. The Interview Scenario

An e-commerce company has approximately 4 TB of order data. Write volume is increasing every month. Analysts repeatedly query the most recent 7-30 days of orders, and customer-facing APIs must continue operating when a database node fails.

- Query-access problem: recent data should be found without unnecessary work across old data.
- Availability problem: a server failure must not become a prolonged application outage.
- Capacity problem: future growth may eventually exceed what one PostgreSQL server can economically handle.

The architecture should therefore be evaluated in layers: partition the large table, provide replication for availability/read scale, and introduce sharding only when horizontal distribution is justified.

3. Partitioning - Managing Very Large Tables

PostgreSQL declarative partitioning splits one logical table into smaller physical pieces. PostgreSQL 18 supports RANGE, LIST, and HASH partitioning. For order data where most queries are date-oriented, RANGE partitioning on order_date is usually the first design to evaluate.

```
CREATE TABLE orders (  
    order_id      bigint,  
    customer_id   bigint,  
    order_date     date NOT NULL,  
    order_status  varchar(30),  
    order_total    numeric(12,2)  
) PARTITION BY RANGE (order_date);  
  
CREATE TABLE orders_2026_08  
PARTITION OF orders  
FOR VALUES FROM ('2026-08-01') TO ('2026-09-01');
```

3.1 Partition Pruning

When query predicates match the partitioning strategy, PostgreSQL can exclude partitions that cannot contain qualifying rows. A query for August orders can therefore avoid scanning unrelated monthly partitions.

```
SELECT *  
FROM orders  
WHERE order_date >= DATE '2026-08-01'  
      AND order_date <  DATE '2026-09-01';
```

3.2 DBA Benefits

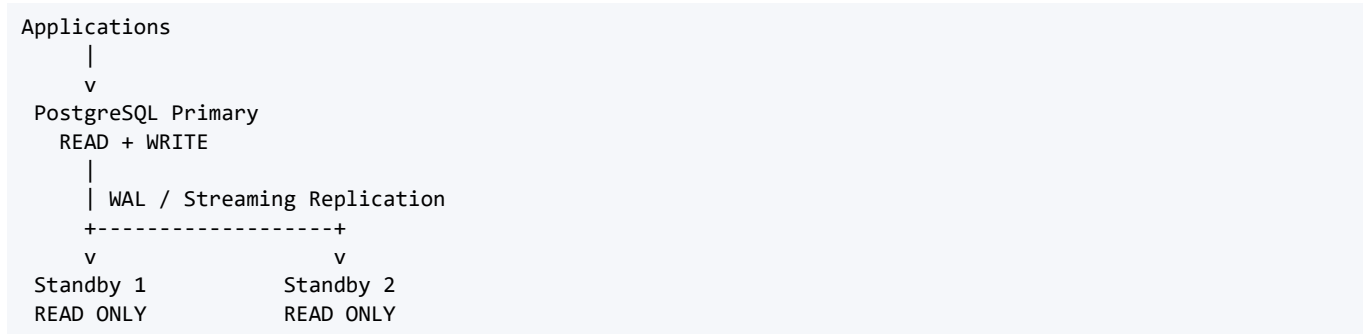
- Recent-data queries can touch a smaller working set when pruning applies.
- Historical partitions can be detached or dropped as part of retention and archival workflows.
- Indexes are physically maintained on individual partitions.
- Bulk lifecycle operations can be easier than operating on one monolithic table.

3.3 What Partitioning Does Not Solve

Important: Partitioning does not automatically distribute the database across multiple PostgreSQL servers. A partitioned table may still live entirely in one PostgreSQL cluster and remain subject to that server's CPU, memory, storage, I/O, and write limits.

4. Replication - High Availability and Read Scale

PostgreSQL supports standby-server architectures in which WAL changes from a primary are transferred to standby servers. A standby can be promoted when the primary fails. A hot standby can also accept read-only queries while recovery is active.



4.1 Availability

Replication provides another copy of the database state from which service can be restored. PostgreSQL documents failover and standby promotion, but production-grade automatic failure detection, routing, fencing, and orchestration are separate architectural concerns.

4.2 Read Scaling

Suitable read-only workloads such as dashboards and reporting can be directed to hot standbys, reducing competition with primary write traffic. This must be designed with workload behavior and recovery conflicts in mind.

4.3 Replication Lag and Read-After-Write

Streaming replication is asynchronous by default. A transaction may commit on the primary before the corresponding WAL has been replayed on a standby. Applications that write an order and immediately read it from a replica must account for possible stale reads.

Design Rule: Critical read-after-write flows should use an appropriate consistency strategy, which may include reading from the primary.

4.4 Replication Is Not Horizontal Write Scaling

Adding more read replicas does not create multiple independent write primaries. The primary still generates the changes that replicas receive. Replication therefore primarily addresses availability and read capacity rather than unlimited write scaling.

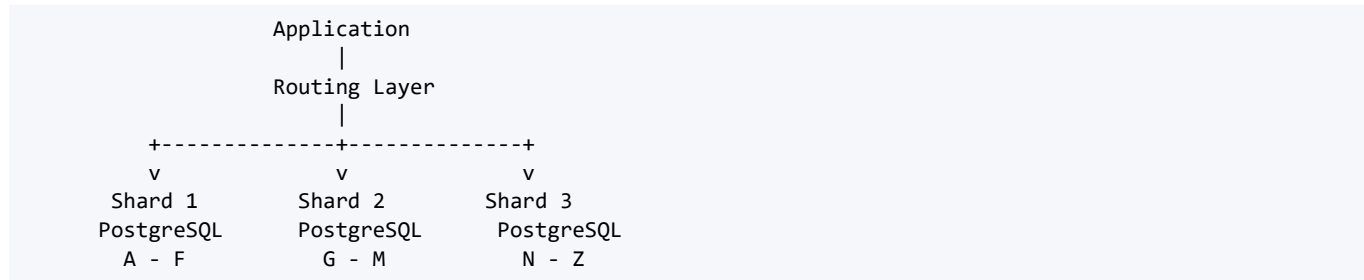
5. Physical vs Logical Replication

Area	Physical Replication	Logical Replication
Replication unit	Physical database-cluster changes/WAL	Data objects and row changes
Typical topology	Primary to standby	Publisher to subscriber
Read-only standby	Supported with hot standby	Subscriber is a normal PostgreSQL database
Fine-grained table selection	Not the main model	Yes - publications/subscriptions
Cross-major-version use	Not its typical role	A documented logical-replication use case

PostgreSQL logical replication uses a publish/subscribe model. It is useful for selected-table replication, consolidation, migrations between major versions, and other fine-grained replication patterns. It should not be confused with transparent automatic sharding.

6. Sharding - Horizontal Distribution Across Servers

Sharding distributes different subsets of data across multiple database nodes. It becomes relevant when a single database server is the fundamental storage, compute, I/O, or write-throughput bottleneck and scaling up is no longer sufficient or desirable.



PostgreSQL Accuracy: Core PostgreSQL does not provide a single built-in transparent automatic distributed-sharding engine. PostgreSQL provides building blocks such as foreign tables and `postgres_fdw` for accessing external PostgreSQL servers; distributed sharding normally requires additional architecture, extensions/products, or application routing.

7. Choosing a Shard Key

The shard key controls where rows are placed and how efficiently requests can be routed. `customer_id` or `tenant_id` may be appropriate when most operations are naturally scoped to a customer or tenant. Hashing can help spread traffic more evenly than simple alphabetical or range boundaries.

- Prefer a key used frequently in request routing and filtering.
- Aim for balanced data volume and transaction traffic.
- Avoid keys that create a small number of extremely hot shards.
- Evaluate joins and transactions that cross shard boundaries.
- Plan for future rebalancing before the first shard becomes full.

8. Operational Cost of Sharding

Sharding trades single-node limits for distributed-system complexity. The design must explicitly handle:

- Cross-shard joins and aggregates
- Distributed transactions
- Query routing
- Hot shards and skew
- Shard rebalancing
- Global identifiers and uniqueness
- Schema changes across shards
- Backup and point-in-time recovery strategy
- Monitoring and connection management
- Failover for each shard

9. Partitioning vs Sharding

Dimension	Partitioning	Sharding
Main idea	Split one logical table into physical partitions	Split data/workload across database nodes
Native PostgreSQL	Yes: RANGE, LIST, HASH	No single transparent automatic distributed-sharding engine
Primary benefit	Manageability and pruning	Horizontal capacity
Cross-node complexity	Normally none	Potentially significant

Memory Aid: Partitioning divides a table. Sharding distributes the workload across database nodes.

10. Sharding vs Replication

Replication and sharding differ in what resides on each node.

```

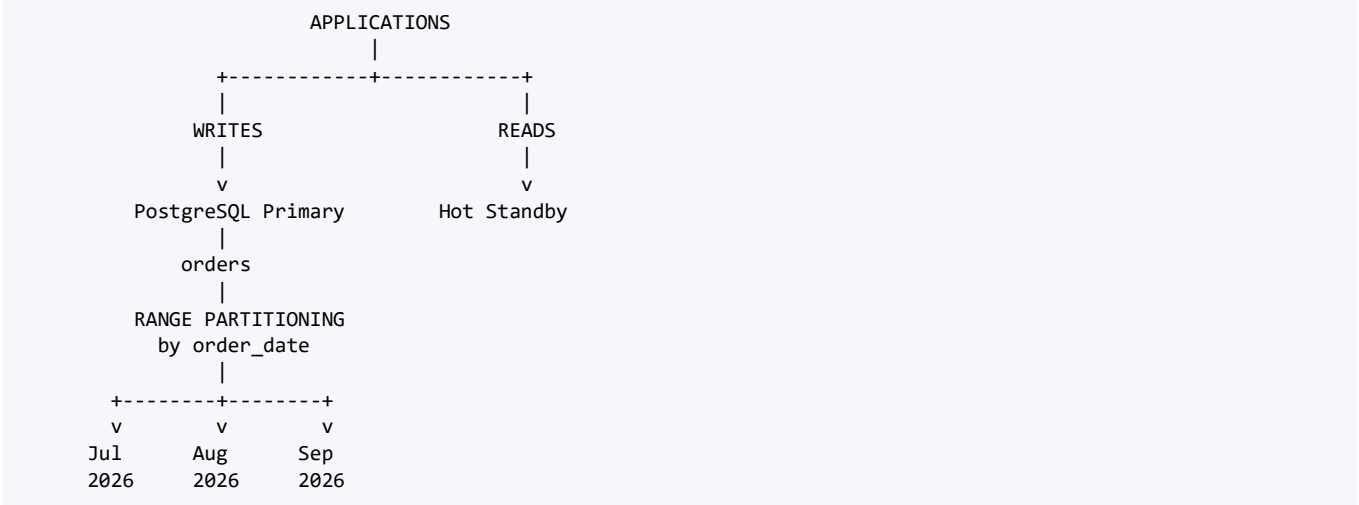
REPLICATION
Primary      Replica 1      Replica 2
A-Z          A-Z           A-Z
Same changes are copied.

SHARDING
Shard 1      Shard 2      Shard 3
A-F          G-M          N-Z
Different subsets are distributed.
  
```

Memory Aid: Replication copies data. Sharding distributes data.

11. Recommended Combined Architecture

For the 4 TB e-commerce scenario, the first architecture to evaluate is partitioning plus replication. Sharding should be introduced only after measurement shows that one properly sized and tuned PostgreSQL server has become the fundamental capacity bottleneck.



At larger scale, each shard can itself use partitioning and replication, producing a layered design: sharding for horizontal capacity, partitioning for local data management, and replication for availability.

12. Production Evolution Path

Stage 1 - Partition

Partition the orders table according to a proven access pattern, such as monthly RANGE partitions on order_date.

Stage 2 - Replicate

Add standby servers for HA and suitable read-only workloads. Define RPO/RTO and failover procedures.

Stage 3 - Measure

Observe CPU, memory, I/O, storage growth, write throughput, connection pressure, query latency, maintenance windows, and replica lag.

Stage 4 - Shard when justified

If a single node becomes the architectural limit, introduce horizontal distribution with a carefully chosen shard key.

Stage 5 - Protect each shard

Apply partitioning and HA/replication independently within each shard where required.

13. PostgreSQL DBA Decision Matrix

Requirement	Partitioning	Replication	Sharding
Manage a very large table	Strong fit	Not primary purpose	Possible, but higher complexity
Partition pruning	Yes	No	Depends on implementation
Historical retention lifecycle	Strong fit	No	Per-shard design required
High availability	No by itself	Strong fit	Requires HA per shard
Read scaling	No by itself	Strong fit	Possible across shards
Horizontal storage scaling	No	No	Strong fit
Horizontal write distribution	No	No	Strong fit
Native PostgreSQL capability	Yes	Yes	Building blocks; no single automatic engine
Operational complexity	Low-Medium	Medium	High

14. Five Architecture Questions

- 1. What is actually slow?** Identify whether the pressure is table scanning, CPU, memory, storage, I/O, reads, writes, or maintenance.
- 2. Are reads or writes the bottleneck?** Replicas can help read-heavy workloads; write saturation needs a different design.
- 3. Can one PostgreSQL server still handle the workload?** Large database size alone is not enough evidence to shard.
- 4. What are the availability targets?** Define RTO/RPO, failover behavior, and acceptable data-loss exposure.
- 5. Is there a clean data-distribution key?** `order_date` is often a partition key; `customer_id` or `tenant_id` may be shard-key candidates depending on access patterns.

15. Interview-Ready Answer

Suggested Answer:

I would not choose only one because partitioning, replication, and sharding solve different bottlenecks. For the 4 TB orders table, I would first evaluate RANGE partitioning by order date because analysts mostly query recent orders.

For availability, I would use PostgreSQL streaming replication with standby servers and design the failover/routing layer around it. I would not shard purely because the database is 4 TB. I would first measure whether a properly sized and tuned primary can continue handling storage and writes. If one server becomes the fundamental capacity limit, I would then evaluate sharding using a key such as `customer_id` or `tenant_id`.

At larger scale, the techniques can be combined: partitioning manages data locally, replication provides availability/read capacity, and sharding distributes storage and write workload across nodes.

16. PostgreSQL-Specific Accuracy Notes

- PostgreSQL 18 provides native declarative RANGE, LIST, and HASH partitioning.
- Partition pruning can exclude irrelevant partitions when query conditions permit.
- PostgreSQL HA documentation covers standby servers, streaming replication, synchronous replication, failover, and hot standby.
- Asynchronous replication can expose applications to replication lag and stale reads.
- Logical replication uses publications and subscriptions and supports fine-grained data replication.
- `postgres_fdw` accesses tables on external PostgreSQL servers and can be used as a building block in multi-server designs.
- Core PostgreSQL should not be described as a transparent, automatic distributed-sharding database engine.

17. Credits and Attribution

Original Author / Concept Creator: [Shubham Srivastava](#)

Role: Principal Data Engineer @ Microsoft CoreAI | ex-Amazon | Data Engineering

Original Post: [Partitioning vs Sharding vs Replication post](#)

Image Credit: The Partitioning vs Sharding vs Replication visual reproduced in this document is credited to Shubham Srivastava / the original LinkedIn post supplied for this article.

Technical Expansion: PostgreSQL-specific explanations, examples, architecture guidance, and validation were developed for this technical document using PostgreSQL 18 official documentation.

18. Official PostgreSQL References

- **PostgreSQL 18 - Table Partitioning**
<https://www.postgresql.org/docs/18/ddl-partitioning.html>
- **PostgreSQL 18 - High Availability, Load Balancing, and Replication**
<https://www.postgresql.org/docs/18/high-availability.html>
- **PostgreSQL 18 - Log-Shipping Standby Servers / Streaming Replication**
<https://www.postgresql.org/docs/18/warm-standby.html>
- **PostgreSQL 18 - Hot Standby**
<https://www.postgresql.org/docs/18/hot-standby.html>
- **PostgreSQL 18 - Logical Replication**
<https://www.postgresql.org/docs/18/logical-replication.html>
- **PostgreSQL 18 - postgres_fdw**
<https://www.postgresql.org/docs/18/postgres-fdw.html>

Validation Statement: The PostgreSQL-specific feature descriptions in this document were checked against PostgreSQL 18 official documentation. Architecture recommendations are presented as design guidance and should be validated against the application's workload, RPO/RTO, consistency requirements, and operational constraints.